

Algoritmos e Programação I

Prof. Dr. Amaury Antônio de
Castro Junior

Módulo 4 - Modularização

Unidade 2 - Retorno e passagem de parâmetros

O que já estudamos?

- Tipos simples (números, caracteres) e comandos básicos de manipulação de dados em Python e suas estruturas de controle de fluxo (condicionais e laços);
- *Strings* e tipos de dados compostos (vetores e matrizes);
- Fundamentos e benefícios da modularização;
- Sintaxe e regras da definição de funções em Python;
- Pequenos exemplos de definição e uso de funções para resolver problemas simples.

Detalhando o retorno de funções

- Uma função pode devolver um resultado (dado) para o programa que a chamou;
- Essa devolução é importante para que o programa principal utilize o valor processado pela sub-rotina em outros cálculos ou decisões;
- Para realizar essa operação, utiliza-se a palavra-chave **return** seguida da variável ou do valor desejado.

Exemplo de função com retorno

```
# Definição da função
def calcula_soma(a, b):
    soma = a + b
    return soma # Envia o resultado de volta ao programa

principal

# Programa principal
resultado = calcula_soma(5, 10) # 'resultado' recebe o valor
15
```

O comando *return* e o fluxo de execução

- **Encerramento imediato:** assim que o comando *return* é executado, a função termina instantaneamente, ignorando qualquer código que venha depois dele dentro do bloco;
- **Múltiplos pontos de retorno:** é comum usar o *return* dentro de estruturas condicionais (*if/else*). A função será encerrada assim que encontrar o primeiro *return* válido.

Exemplo

```
# Fluxo de execução da função com dois returns
def verifica_valor(x):
    if x < 0:
        return -1 # Encerra a função se o valor for negativo
    print("Este texto só aparece se x >= 0")
    return 0 # Encerra a função caso contrário
```

Como aproveitar o valor retornado?

- O valor devolvido por uma função pode ser utilizado de diversas maneiras no programa principal:
 1. Atribuição a uma variável: armazena o dado para uso posterior:
Exemplo: `s = soma(a, b)`
 2. Expressão aritmética: o retorno entra direto em um cálculo
Exemplo: `total = 10 * soma(a, b)`

Como aproveitar o valor retornado?

- O valor devolvido por uma função pode ser utilizado de diversas maneiras no programa principal:
3. Condição de decisão: o resultado é testado em alguma estrutura condicional ou de repetição. Exemplo: `if`
`soma(a, b) > 0:`
 4. Como Iterador: caso a função retorne uma lista ou coleção, pode ser usada em um `for`.

Retornando múltiplos valores

- O Python permite que uma única função retorne vários valores simultaneamente, separados por vírgula no comando *return*;
- Empacotamento em tupla: o interpretador agrupa esses valores em uma tupla (conjunto imutável);
- Desempacotamento: na chamada da função, é possível atribuir cada valor retornado diretamente a variáveis separadas.

Exemplo de retorno de múltiplos valores

```
# Função que retorna dois valores
def calcular_operacoes(x, y):
    soma = x + y
    subtracao = x - y
    return soma, subtracao # Retorna dois valores

# Atribuição múltipla
res_soma, res_sub = calcular_operacoes(10, 5)
```

Funções sem retorno

- Nem toda função precisa devolver um dado; algumas existem apenas para executar ações, como exibir mensagens na tela;
- Se o comando *return* não for utilizado, a função termina naturalmente após a execução de sua última linha;
- O Valor **None**: Se você tentar atribuir o "resultado" de uma função sem retorno a uma variável, ela receberá o valor especial **None** (que representa a ausência de valor).

Exemplo de função sem retorno

```
# Exemplo de função sem retorno
def imprime_mensagem(texto):
    print(f"Aviso: {texto}") # Apenas executa uma ação
    # Não tem comando "return"

# 'resultado' será None
resultado = imprime_mensagem("Erro de sistema")
```

Funções e escopo de variáveis

- **Escopo:** refere-se à visibilidade de variáveis em funções.
- **Variáveis locais:**
 1. Criadas dentro da função e destruídas ao fim dela;
 2. Visíveis apenas internamente.
- **Variáveis globais:**
 1. Criadas fora das funções (geralmente, no programa principal).
 2. Visíveis por todo o programa.

Exemplo ilustrativo de escopo

Variável Global - É definida fora de qualquer função e pode ser acessada por todo o programa.

```
variavel_global = "Eu sou Global (fora da função)"
```

```
def minha_funcao():
```

```
    # Variável Local - É definida dentro da função e só existe enquanto a função está em execução.
```

```
    variavel_local = "Eu sou Local (dentro da função)"
```

```
    print(f"Dentro da função (Local): {variavel_local}")
```

```
    print(f"Dentro da função (Global): {variavel_global}")
```

```
    # Para alterar uma variável global dentro de uma função, é necessário usar a palavra-chave 'global'
```

```
    # (como no exemplo abaixo). Sem 'global', Python criaria uma nova variável LOCAL com o mesmo nome.
```

```
    # Exemplo de alteração de variável global (comentado para este exemplo):
```

```
    #     global variavel_global
```

```
    #     variavel_global = "Global foi alterada dentro da função"
```

```
minha_funcao() # Chamada da função
```

```
# Acesso à variável global fora da função
```

```
print(f"Fora da função (Global): {variavel_global}")
```

```
# Tentativa de acesso à variável local (irá gerar um erro, pois ela não existe mais)
```

```
# print(f"Fora da função (Local): {variavel_local}")
```

Exemplo ilustrativo de escopo

Variável Global - É definida fora de qualquer função e pode ser acessada por todo o programa.

```
variavel_global = "Eu sou Global (fora da função)"
```

```
def minha_funcao():
```

```
    # Variável Local - É definida dentro da função e só existe enquanto a função está em execução.
```

```
    variavel_local = "Eu sou Local (dentro da função)"
```

```
    print(f"Dentro da função (Local): {variavel_local}")
```

```
    print(f"Dentro da função (Global): {variavel_global}")
```

```
    # Para alterar uma variável global dentro de uma função, é necessário usar a palavra-chave 'global'
```

```
    # (como no exemplo abaixo). Sem 'global', Python criaria uma nova variável LOCAL com o mesmo nome.
```

```
    # Exemplo de alteração de variável global (comentado para este exemplo):
```

```
    #     global variavel_global
```

```
    #     variavel_global = "Global foi alterada dentro da função"
```

```
minha_funcao() # Chamada da função
```

```
# Acesso à variável global fora da função
```

```
print(f"Fora da função (Global): {variavel_global}")
```

```
# Tentativa de acesso à variável local (irá gerar um erro, pois ela não existe mais)
```

```
# print(f"Fora da função (Local): {variavel_local}")
```


Exemplo ilustrativo de escopo

Variável Global - É definida fora de qualquer função e pode ser acessada por todo o programa.

```
variavel_global = "Eu sou Global (fora da função)"
```

```
def minha_funcao():
```

```
    # Variável Local - É definida dentro da função e só existe enquanto a função está em execução.
```

```
    variavel_local = "Eu sou Local (dentro da função)"
```

```
    print(f"Dentro da função (Local): {variavel_local}")
```

```
    print(f"Dentro da função (Global): {variavel_global}")
```

```
    # Para alterar uma variável global dentro de uma função, é necessário usar a palavra-chave 'global'  
    # (como no exemplo abaixo). Sem 'global', Python criaria uma nova variável LOCAL com o mesmo nome.  
    # Exemplo de alteração de variável global (comentado para este exemplo):
```

```
    #     global variavel_global
```

```
    #     variavel_global = "Global foi alterada dentro da função"
```

```
minha_funcao() # Chamada da função
```

```
# Acesso à variável global fora da função
```

```
print(f"Fora da função (Global): {variavel_global}")
```

```
# Tentativa de acesso à variável local (irá gerar um erro, pois ela não existe mais)
```

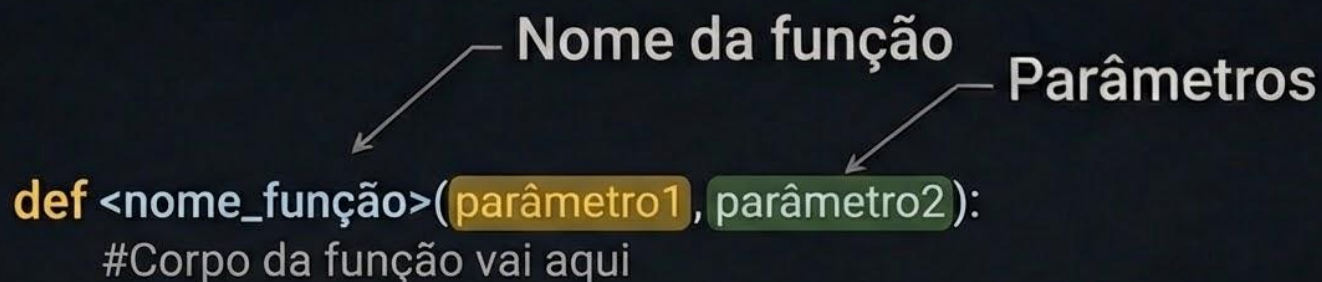
```
# print(f"Fora da função (Local): {variavel_local}")
```

Variáveis globais X Variáveis locais

- **CUIDADO:**
 1. O uso de variáveis globais dificulta a compreensão do código;
 2. Torna a correção de erros no programa muito mais complexa;
 3. Encontrar um erro no valor de uma global pode ser difícil:
- Recomenda-se usar variáveis locais nas funções sempre que possível e passar os valores necessários à função como parâmetros.

Passagem de parâmetros ou argumentos

- Usado para enviar dados (valores) para dentro de funções;
- São definidos na mesma linha do cabeçalho da função.



The diagram shows the syntax of a function definition: `def <nome_função>(parâmetro1, parâmetro2):`. An arrow points from the label "Nome da função" to the text "<nome_função>". Another arrow points from the label "Parâmetros" to the text "(parâmetro1, parâmetro2)". Below the function signature, the text "#Corpo da função vai aqui" is shown.

```
def <nome_função>(parâmetro1, parâmetro2):  
    #Corpo da função vai aqui
```

Fonte: autoria própria

Tipos de passagem de parâmetros

- **Passagem por valor (ou cópia):** o valor é copiado; a variável original não muda (padrão para tipos simples em Python);
- **Passagem por referência:** O que é passado é o endereço de memória. Assim, alterações na função se refletem no programa principal;
- **Parâmetros *default*:** É definido um valor padrão para quando o usuário não fornece o dado.

Exemplo ilustrativo de passagem de parâmetros (por valor e *default*)

```
def calcular_gorjeta(valor, percentual=10):  
    return valor * percentual/100  
  
print(calcular_gorjeta(400))      # Usa 10% (default)  
print(calcular_gorjeta(400, 5))  # Usa 5%
```

Exemplo ilustrativo de passagem de parâmetros (por valor)

```
# 1. FUNÇÃO POR VALOR (Parâmetro simples)
# O valor da variável é copiado. Alterações aqui NÃO afetam o original.
def aplicar_desconto_valor(preco):
    preco = preco * 0.9 # Tenta aplicar 10% de desconto
    print(f"Dentro da função (valor): R$ {preco}")

# --- PROGRAMA PRINCIPAL ---

# Teste 1: Passagem por Valor
valor_original = 100.0
print(f"Antes da função (valor): R$ {valor_original}")
aplicar_desconto_valor(valor_original)
print(f"Depois da função (valor): R$ {valor_original} <- Não mudou!")
```

Exemplo ilustrativo de passagem de parâmetros (por valor)

```
# 1. FUNÇÃO POR VALOR (Parâmetro simples)
# O valor da variável é copiado. Alterações aqui NÃO afetam o original.
def aplicar_desconto_valor(preco):
    preco = preco * 0.9 # Tenta aplicar 10% de desconto
    print(f"Dentro da função (valor): R$ {preco}")
```

Saída:

```
# --- PROGRAMA PRINCIPAL ---
```

```
# Teste 1: Passagem por Valor
```

```
valor_original = 100.0
```

```
print(f"Antes da função (valor): R$ {valor_original}")
```

```
aplicar_desconto_valor(valor_original)
```

```
print(f"Depois da função (valor): R$ {valor_original} <- Não mudou!")
```

```
Antes da função (valor): R$ 100.0
Dentro da função (valor): R$ 90.0
Depois da função (valor): R$ 100.0 <- Não mudou!
```

Exemplo ilustrativo de passagem de parâmetros (por referência)

```
# 2. FUNÇÃO POR REFERÊNCIA (Usando uma Lista)
# O endereço do vetor é copiado. Alterações aqui AFETAM o original.
def aplicar_desconto_referencia(lista_preco):
    lista_preco = lista_preco * 0.9 # Altera o conteúdo original
    print(f"Dentro da função (referência): R$ {lista_preco}")

# --- PROGRAMA PRINCIPAL ---

# Teste 2: Passagem por Referência
lista_original = [100.0]
print(f"Antes da função (referência): R$ {lista_original}")
aplicar_desconto_referencia(lista_original)
print(f"Depois da função (referência): R$ {lista_original} <- O VALOR FOI ALTERADO!")
```


Exemplo ilustrativo de passagem de parâmetros (por referência)

```
# 2. FUNÇÃO POR REFERÊNCIA (Usando uma Lista)
# O endereço do vetor é copiado. Alterações aqui AFETAM o original.
def aplicar_desconto_referencia(lista_preco):
    lista_preco = lista_preco * 0.9 # Altera o conteúdo original
    print(f"Dentro da função (referência): R$ {lista_preco}")
```

Saída:

```
# --- PROGRAMA PRINCIPAL ---
```

```
# Teste 2: Passagem por Referência
```

```
lista_original = [100.0]
```

```
print(f"Antes da função (referência): R$ {lista_original}")
```

```
aplicar_desconto_referencia(lista_original)
```

```
print(f"Depois da função (referência): R$ {lista_original} <- O VALOR FOI ALTERADO!")
```

```
Antes da função (referência): R$ [100.0]
Dentro da função (referência): R$ [90.0]
Depois da função (referência): R$ [90.0] <- O VALOR FOI ALTERADO!
```

Resumindo

- Python usa passagem de parâmetro por valor por padrão:
 1. Faz cópia do valor da variável original para o parâmetro da função;
 2. Valor original não é modificado por eventuais alterações dentro da função.

Resumindo

- Exceção: **vetores** (listas) funcionam de forma diferente, pois o valor de um **vetor** é seu **endereço**, ou seja, o acesso é indireto. Vamos ver como isso funciona:
 1. O que é copiado é o endereço da memória (referência), e, portanto, qualquer alteração é refletida no valor original armazenado.

Dicas e próximos passos

- Implemente e teste os códigos apresentados nessa aula em IDEs e interfaces interativas:
 1. IDEs: Thonny, PyCharm, VSCode;
 2. Interfaces interativas: IDLE, Jupyter Notebook, Google Colab.
- Aprender algoritmos exige empenho, dedicação e muitos... muitos exercícios;
- Leia os materiais indicados como referências;

Dicas e próximos passos

- Busque outras fontes;
- Faça pesquisas e exercícios;
- Comprometa-se com o curso e com a disciplina.

Referências

BANIN, S. L. **Python 3: conceitos e aplicações - uma abordagem didática**. São Paulo: Erica, 2018. ISBN 9788536530253. [Disponível na Biblioteca Digital da UFMS.](#)

MANZANO, J. A. N. G. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29 ed. São Paulo: Erica, 2019. ISBN 9788536531472. [Disponível na Biblioteca Digital da UFMS.](#)

PERKOVIC, L. **Introdução à computação usando Python: um foco no desenvolvimento de aplicações**. Rio de Janeiro: LTC, 2016. ISBN 9788521630937. [Disponível na Biblioteca Digital da UFMS.](#)

WENTWORTH, P.; ELKNER, J.; DOWNEY, A. B.; MEYERS, C. **How to think like a computer scientist: learning with Python 3**. [S.L.: S. N.], 2012. Disponível em: <https://link.ufms.br/Vow0N>. Acesso em 05 mai. 2025.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

